

Advanced Cisco Networking Academy - Hosting a Local AI Model with a Web Interface

Jonathan Kleven
CCNA



Purpose

In recent years, AI has experienced a spike in popularity as its power and accessibility have increased exponentially. This lab walks through how to host an AI locally on any computer and access it through a web interface. Additionally, this lab will demonstrate how local AI can be used for more than just a simple chatbot by implementing it into a Python script and giving the AI the ability to do far more than just talk.

Background

AI is a broad category that encompasses algorithms and machines that can exhibit human intelligence by learning or communicating like a human. This type of computing has been the goal of computer scientists for decades, and this idea is often considered to have found its roots in a man named Alan Turing.

Turing was a British logician in the early to mid-1900s. He was fascinated by humans' ability to learn, and wanted to find a way to build a machine that could think like a human being. In 1935, Turing described a theoretical machine that could read the instructions and input for any describable machine and replicate the output. In short, it was a general-purpose machine that could simulate other machines if given a set of instructions and some input. This idea was fundamental to the modern computer, which can run any program that it is given. This Universal Turing Machine partially captured Turing's dream, but it wasn't enough. He wanted the machine to be able to learn, essentially writing its own instructions.

After cracking Enigma, the German encryption method used to transmit messages securely during WWII, Alan Turing described a test in his 1950 paper "Computing Machinery and Intelligence" that could be used to identify if a machine had successfully been created to learn, think, and talk like a human by altering its own instructions. This test, famously known as the Turing Test, has a human and a computer respond to an interrogator's questions through a keyboard and a display screen. Given that the computer is allowed to lie and force an incorrect identification, if a sufficient number of interrogators are unable to distinguish between the human and the computer, the machine is considered an intelligent, thinking entity.

This concept caught the interest of many computer scientists, eventually leading to the development of the neural network with Rosenblatt's Perceptron. A neural network is a scaled-down simulation of the human brain, consisting of interconnected nodes arranged in layers that process information by accepting inputs, performing mathematical operations on them, and generating an output. Through a process of training with labeled examples or reinforcement, also known as machine learning, the values within the neural network are updated gradually to

produce better results. The larger the network is, the longer it will take to train, but it will be able to process more complex data with more precise outputs.

As computer scientists experimented with these simple intelligent algorithms, they began to tackle the problem of human language. Alan Turing wanted a machine that could process human language and generate responses that imitated a human. To do this, modern AI applications use algorithms that process text in their own language based on a set of tokens. These tokens are small units of data created by breaking down larger pieces of information. By dividing the data into smaller parts, the AI can identify relationships between tokens, enabling functions like prediction, generation, and reasoning. To limit the amount of data an AI is allowed to process, old tokens are replaced with new tokens once a threshold has been crossed. These AI models are called Large Language Models (LLMs). Each model is made of a certain number of parameters. These parameters are also called weights. As the model is trained, these weights are tuned to create better results based on the input. A greater number of weights often results in “smarter” responses but requires significantly more processing power to train and run. The number of parameters ranges from a few billion in a moderately-sized model to trillions in the latest GPT-5 model.

As LLMs became more powerful, people started putting them through the Turing Test. It wasn't until April 2025 that a model would finally beat it. OpenAI, one of the largest artificial intelligence research and deployment companies, created ChatGPT, a very powerful LLM. After multiple models, their GPT-4.5 model managed to pass the Turing Test with interrogators mistaking the AI for a human over 73% of the time.

Today, these models are available for free online. Using a public AI like ChatGPT that runs on a large server managed by OpenAI comes with many benefits. The responses are often very smart and can be generated fast. Additionally, it requires no setup to use. It's free and readily available. However, this comes with security risks. Many online AI collect data from the prompts and files given to the AI by the user. This data is often sold or given back to the AI to expand the training. If the user enters personal information or asks an AI to analyze a private document, this information could be collected, sold, or processed.

To avoid security risks, modern software allows anyone to run an AI locally. Doing this keeps all data private: it cannot be accessed from outside the server. Local models give the owner complete control over the data collected through prompts, including the ability to erase everything without a trace. This keeps confidential information private.

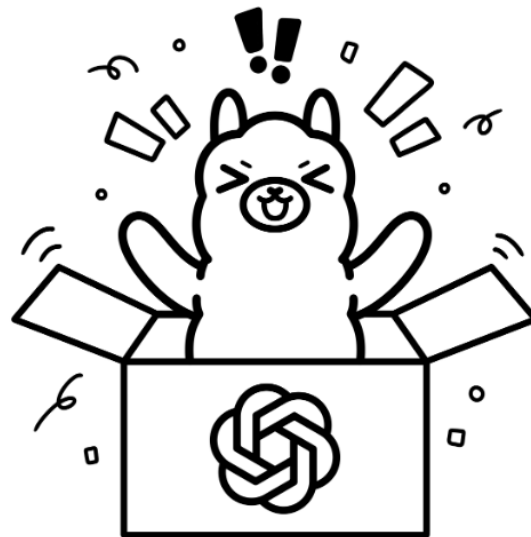
With Ollama or LM Studio to host the models, anyone can run an LLM on their computer. The power of the model will depend heavily on the hardware, with more RAM and VRAM meaning an easier time running larger models. Docker

allows you to package everything, like dependencies, runtime, and configurations, into one container. This ensures things run smoothly across multiple devices and versions. Further, it doesn't require much additional effort to make the model accessible through a web browser. With OpenWebUI, a self-hosted, open-source AI platform that provides a simple web interface, creating a web-based AI is a simple process.

Beyond security, local AI models tend to have lower latency by eliminating the delays from sending data between your computer and a remote server. Additionally, local hosting provides offline access, making the system more robust. Furthermore, the host would be able to fully customize the model and its parameters, allowing for extensive and precise control over the model without limitations or censorship.

Installation

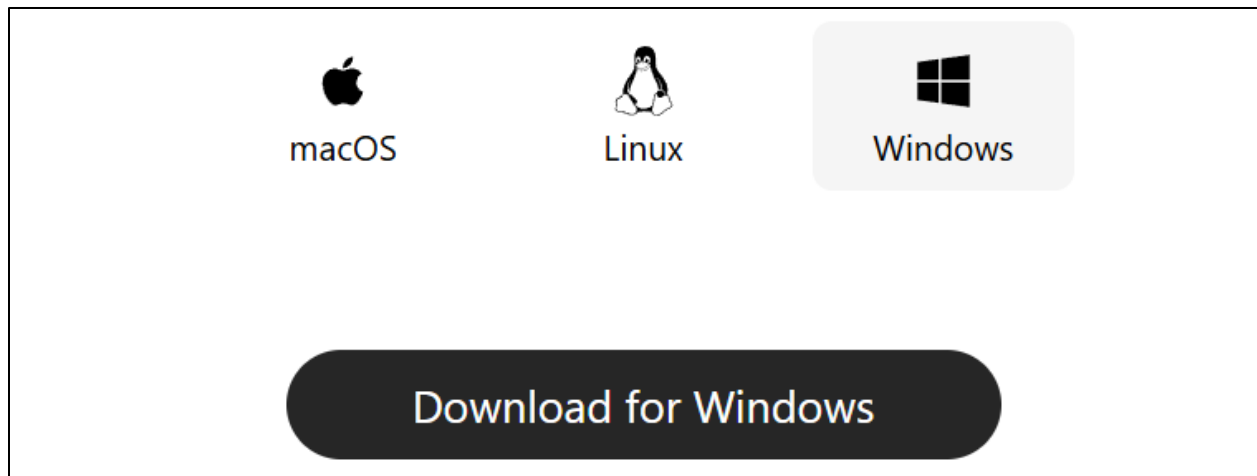
To begin hosting local AIs on your machine, you will first need to install the Ollama software to run the models.



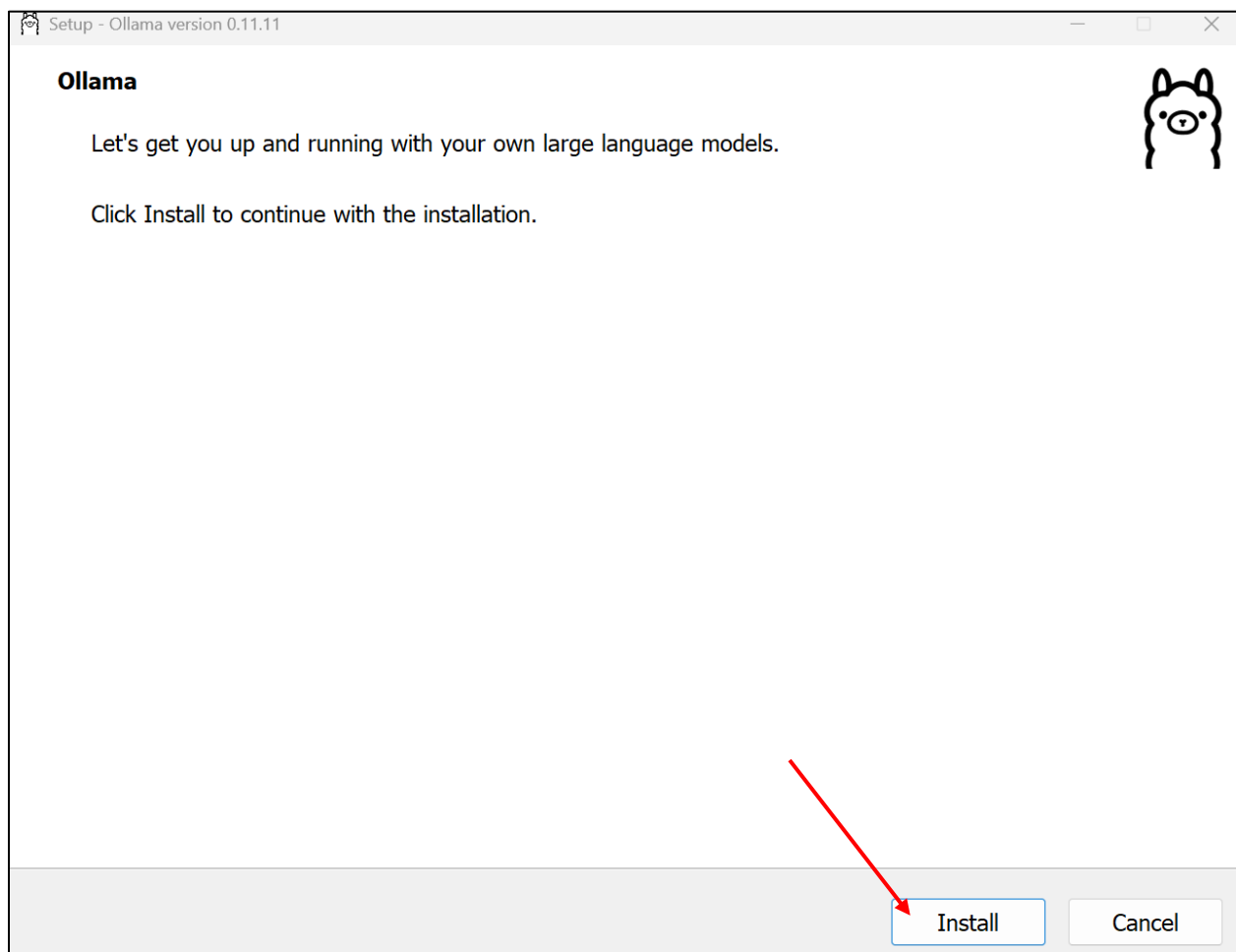
Download

Available for macOS,
Windows, and Linux

Visit ollama.com and click the download button.



Download for Windows and open the executable when it has finished downloading.



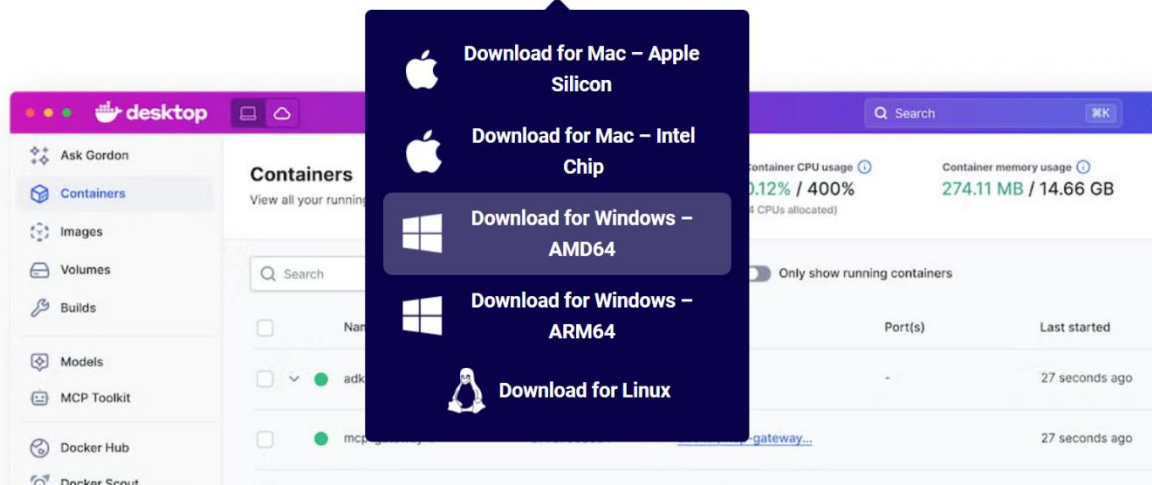
Begin the Ollama installation. This will take some time.

Develop Apps!

Your foundation for secure, intelligent development

Download Docker Desktop

Learn about Docker for AI



While Ollama downloads, go to docker.com and download Docker Desktop for your machine. For Windows 11, download the Windows – AMD64 version.

Configuration

- ☒ Use WSL 2 instead of Hyper-V (recommended)
- ☐ Allow Windows Containers to be used with this installation
- ☒ Add shortcut to desktop

Windows Containers should only be enabled if you understand the risks. For more information, see [our docs](#).

OK

When it downloads, run the executable, ensure “Use WLS2” is checked, and click “OK” to begin the installation.

Installation succeeded

You must restart Windows to complete installation.

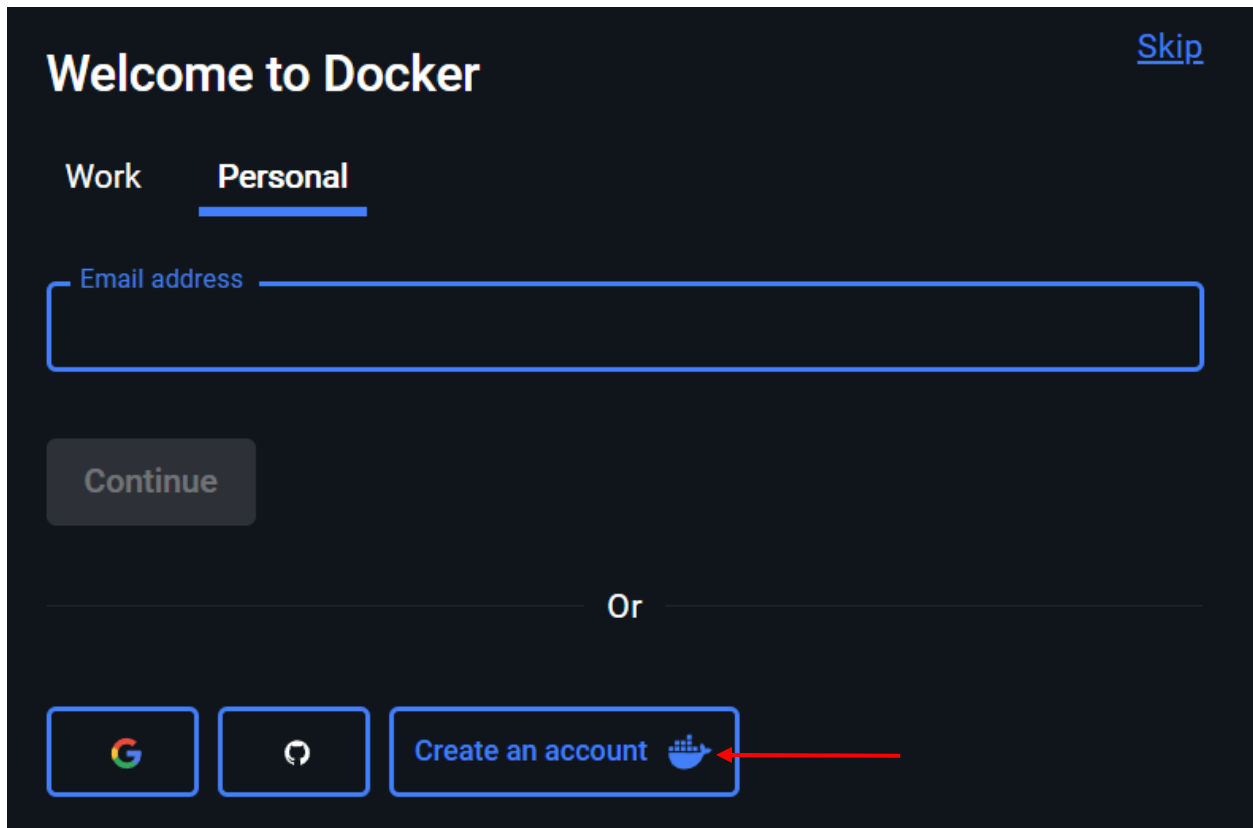
Close and restart

When Docker finishes installing, it may ask you to restart your computer. If it does not, it is recommended to still do so. However, wait until Ollama has finished installing before restarting.


```
C:\WINDOWS\system32\wsl.e x + v
Windows Subsystem for Linux must be updated to the latest version to proceed. You can update by running 'wsl.exe --update'.
For more information please visit https://aka.ms/wslinstall

Press any key to install Windows Subsystem for Linux.
Press CTRL-C or close this window to cancel.
This prompt will time out in 60 seconds.
```

After restarting, you should be prompted with a request to install Windows Subsystem for Linux. Press any key to start.



Welcome to Docker [Skip](#)

Work **Personal**

Email address

Continue

Or

  **Create an account**  

If Docker has not automatically started, find Docker in the Windows apps from the start menu, and create an account.

Work Personal

Email

Email is required.

Username

Use 4 to 30 letters & digits only.

Password 

☐ Send me occasional product updates and announcements.

Sign up

Enter your details and click “Sign Up.”

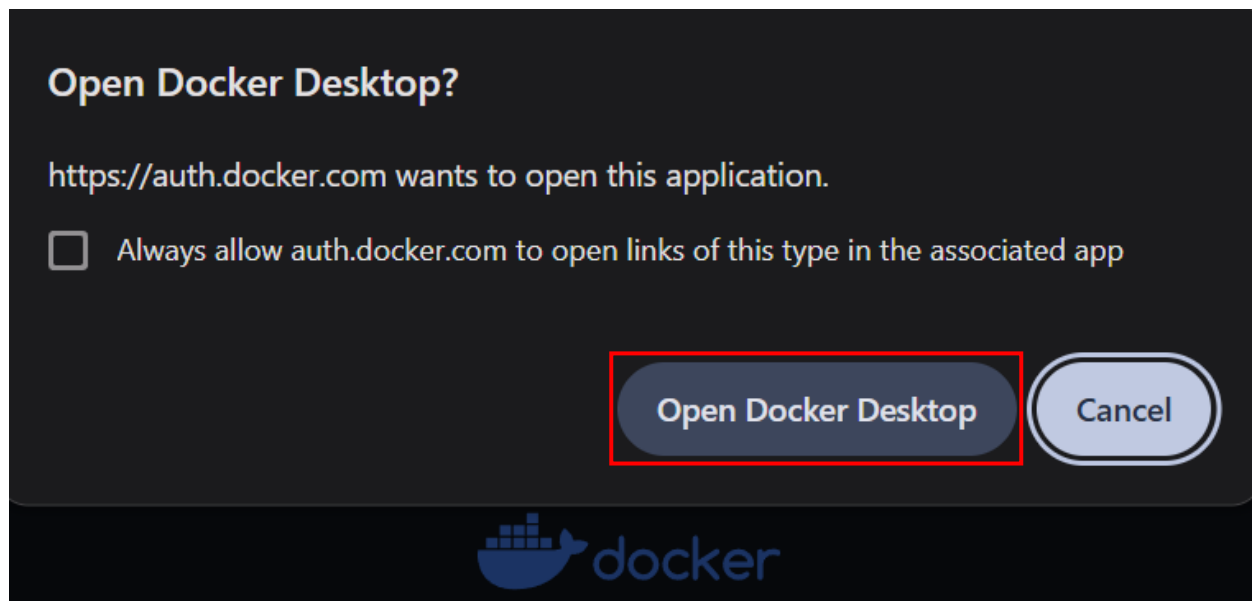
 docker®

Sign in

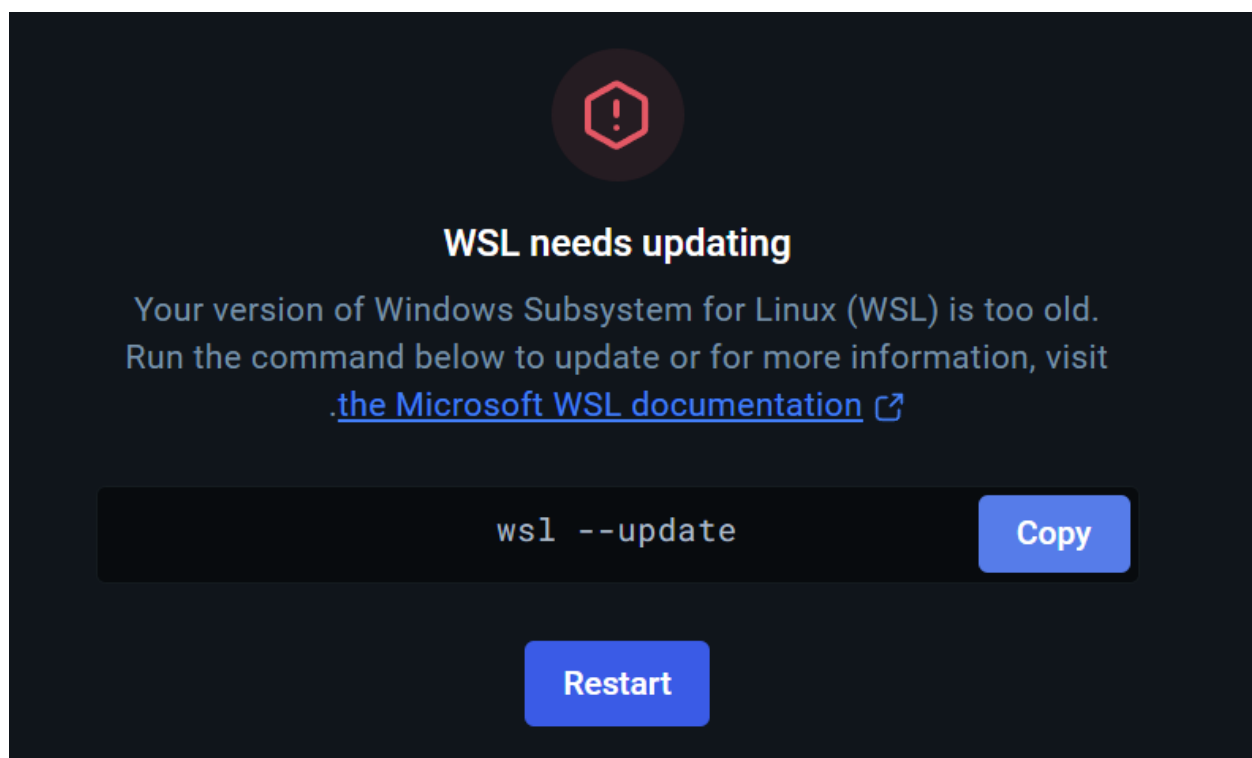
Username or email address*

Continue

After creating an account, sign in to Docker with the account you just created.



After logging in, the web browser will redirect you back to the desktop app.

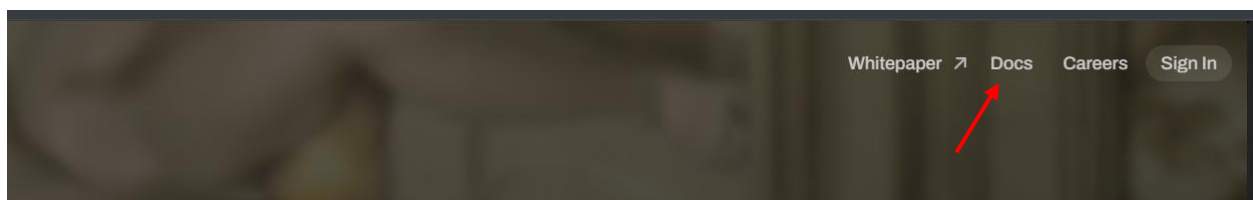


Back in Docker, you may be asked to update WSL. Copy the provided command and open the command prompt.

```
Command Prompt
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Antigua>wsl --update
```

Right-click or press Ctrl+V to paste in the command before pressing Enter. When the update finishes, go back to Docker and click “Restart.”



To set up a web interface quickly for multi-user access across the local network, go to openwebui.com and go to the docs.

Open WebUI Bundled with Ollama

This installation method uses a single container image that bundles Open WebUI with Ollama, allowing for a streamlined setup via a single command. Choose the appropriate command based on your hardware setup:

- **With GPU Support:** Utilize GPU resources by running the following command:

```
docker run -d -p 3000:8080 --gpus=all -v ollama:/root/.ollama -v open-webui:/app
```

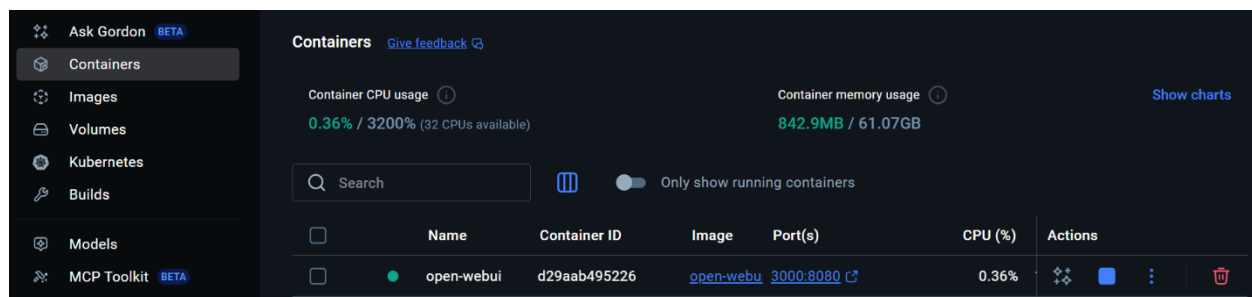
Scroll down to the Open WebUI bundled with Ollama, and find the command labeled for GPU support. Copy this command and go back to the command prompt.

```
C:\Users\Antigua>docker run -d -p 3000:8080 --gpus=all -v ollama:/root/.ollama -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:ollama
```

Right-click or press Ctrl+V to paste the command, and press Enter to run it.

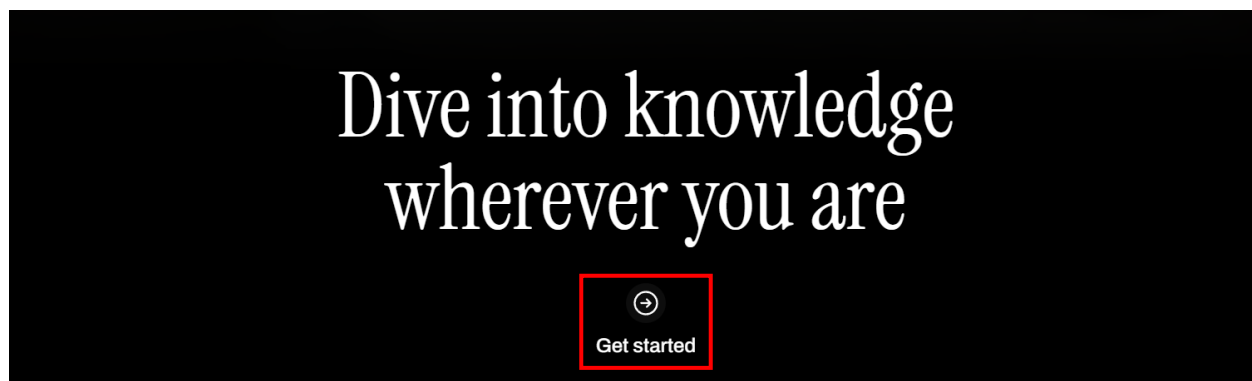
```
ollama: Pulling from open-webui/open-webui
d107e437f729: Pull complete
4f4fb700ef54: Pull complete
49b1c40da033: Pull complete
f1451649e055: Pull complete
427b6e68e4f6: Pull complete
45b71b5dc15d: Pull complete
ce4392d98604: Pull complete
dfcb19d2cc14: Pull complete
1fd1c0f46ae6: Pull complete
4079926b32d5: Pull complete
e07e6da0bc93: Pull complete
b0b6ac0f5373: Pull complete
a830e79592b2: Pull complete
f8043d35c69a: Pull complete
1ce95d9ae66c: Pull complete
659036234d55: Pull complete
```

Wait until all the pulls are completed.



Name	Container ID	Image	Port(s)	CPU (%)	Actions
open-webui	d29aab495226	open-webui	3000:8080	0.36%	[Start/Stop/Restart/Logs/Remove]

Back in Docker, locate the open-webui container under the “Containers” tab. If there is a triangle instead of a square beneath actions, click the triangle to start it. Once started, find the link under the ports and click it, or open a web browser and go to localhost:<port_number>. The default port should be 3000.



In the web interface, click “Get Started.”

Get started with Open WebUI

① Open WebUI does not make any external connections, and your data stays securely on your locally hosted server.

Name

Enter Your Full Name

Email

Enter Your Email

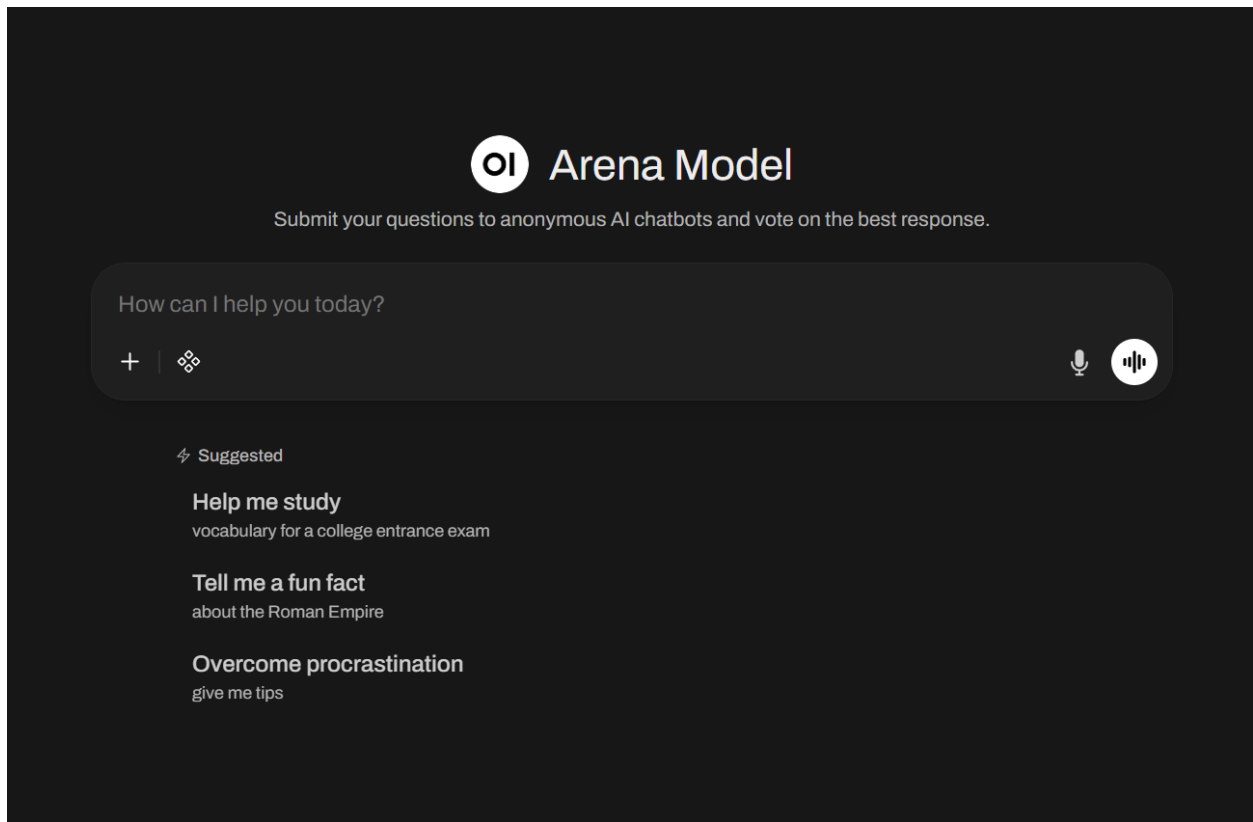
Password

Enter Your Password



Create Admin Account

Create an account that will act as the administrator for the AI.



In the web interface, you should see a classic AI environment where you can chat with an AI. However, there aren't any models loaded. To begin a conversation with an AI, you need to pull models and load them here.



To find models, go back to the Ollama website at ollama.com and click on “Models” at the top of the page.

llava

 LLaVA is a novel end-to-end trained large multimodal model that combines a vision encoder and Vicuna for general-purpose visual and language understanding. Updated to version 1.6.

vision

7b

13b

34b



9.7M Pulls



98 Tags



Updated 1 year ago

From the list, find a model that fits your needs and click on the name.

Models

[View all →](#)

Name	Size	Context	Input
llava:latest	4.7GB	32K	Text, Image
<u>llava:7b</u> latest	4.7GB	32K	Text, Image
llava:13b	8.0GB	4K	Text
llava:34b	20GB	4K	Text

Select a model size based on your hardware limitations.

llava:7b

ollama run llava:7b



9.7M Downloads



Updated 1 year ago

 LLaVA is a novel end-to-end trained large multimodal model that combines a vision encoder and Vicuna for general-purpose visual and language understanding. Updated to version 1.6.

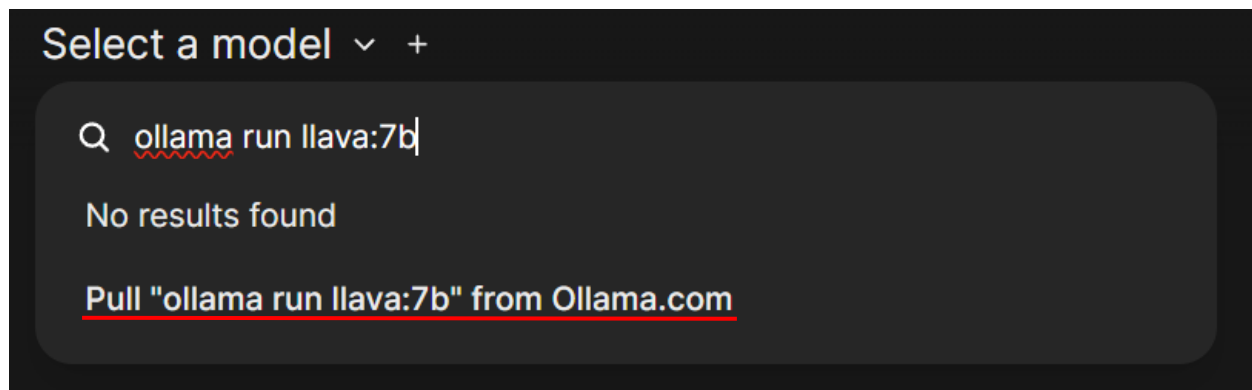
vision

7b

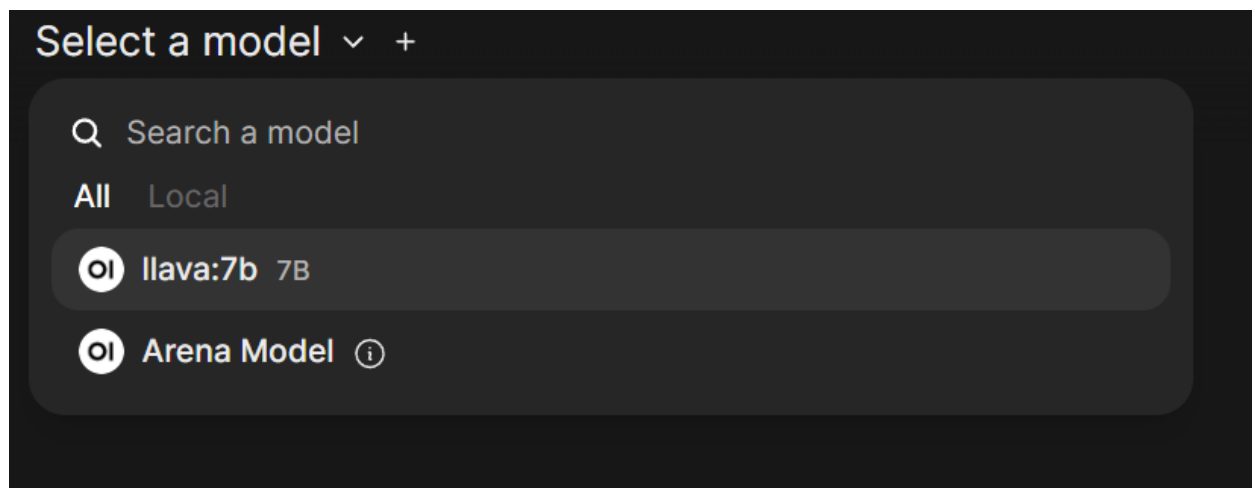
13b

34b

At the top of the page, copy the command and go back to the OpenWebUI interface.



Click "Select a model" and paste the command into the search bar. It will tell you that no results were found, but it will ask if you want to pull the model. Click this option and wait for the model to load.



The model will appear in the list. Click it to begin chatting with it.

Today at 5:58 PM

What are the ingredients and the best temperature + time in the oven to get a crispy bread with a soft core?

 **llava:7b**

To achieve a crispy exterior and a soft interior in a loaf of bread, you need to control both the baking temperature and the baking time. Here's a general guideline:

Ingredients:

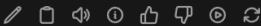
- 4 cups all-purpose flour
- 2 teaspoons salt
- 1 tablespoon sugar (optional)
- 2 1/4 teaspoons active dry yeast
- 1 3/4 cups water
- 2 tablespoons vegetable oil, plus more for brushing

Instructions:

1. In a large bowl, combine the flour, salt, sugar (if using), and yeast. Mix well to combine.
2. Add the water and vegetable oil, and mix until a dough forms. If the dough is too sticky to handle, add more flour, a tablespoon at a time.
3. Turn the dough out onto a floured surface and knead for about 10 minutes, or until it becomes smooth and elastic.
4. Place the dough in a lightly oiled bowl, cover it with a clean kitchen towel, and let it rise in a warm place for about 1 to 1.5 hours, or until it has doubled in size.
5. Preheat your oven to 425°F (220°C).
6. Once the dough has risen, turn it out onto a floured surface and shape it into a loaf. Place the shaped loaf on a baking sheet lined with parchment paper or a silicone baking mat.
7. Brush the top of the loaf with a little more vegetable oil.
8. Bake the loaf for 20-25 minutes, or until it is golden brown and sounds hollow when tapped on the bottom. The internal temperature of the bread should reach 190°F (87°C) to ensure that it is fully cooked.

Tips:

- For a crispier crust, you can also add seeds or nuts to the surface of the dough before baking.
- If you prefer a darker crust, you can brush the top of the loaf with an egg wash (made by whisking an egg with a tablespoon of water) before baking.
- Allow the bread to cool on a wire rack for at least 15-20 minutes before slicing and serving. This will give the bread time to set and prevent it from becoming too soggy.



You are now ready to talk with the AI. Additionally, since the AI is accessible by a web browser, the AI can be used on multiple devices connected to the same network as the host.

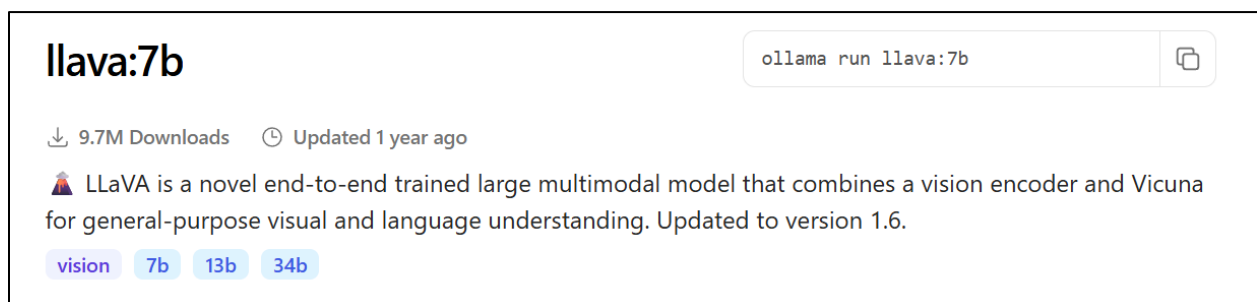
Python Implementation

With the AI running locally, it is possible to use it for far more than an AI chatbot through a prebuilt web interface. With some Python, it is possible to send and receive AI answers in code. This allows for intelligent decision making, quick user pattern recognition, personalization, and more.

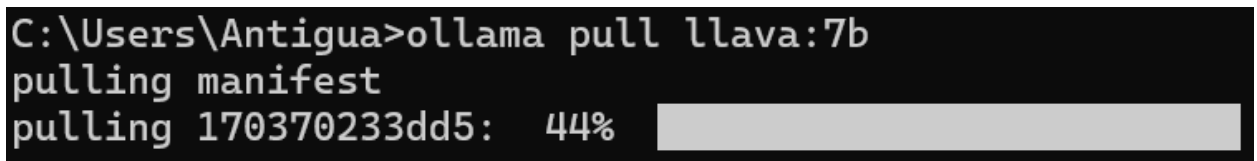
Before coding, we'll start by downloading a model to use. We cannot immediately use the model installed earlier since that model was installed into the Docker container, and this section does not use Docker.



To download a model, go back to the Ollama website and relocate the Models tab.



Find a model that will work the best for your program and memorize the name of the model. In this case, the name of the model is "llava:7b"



Open the command prompt and enter the command "Ollama pull <model_name>". This will download the model to your computer, allowing you to access it through Python with the ollama library.

PyCharm

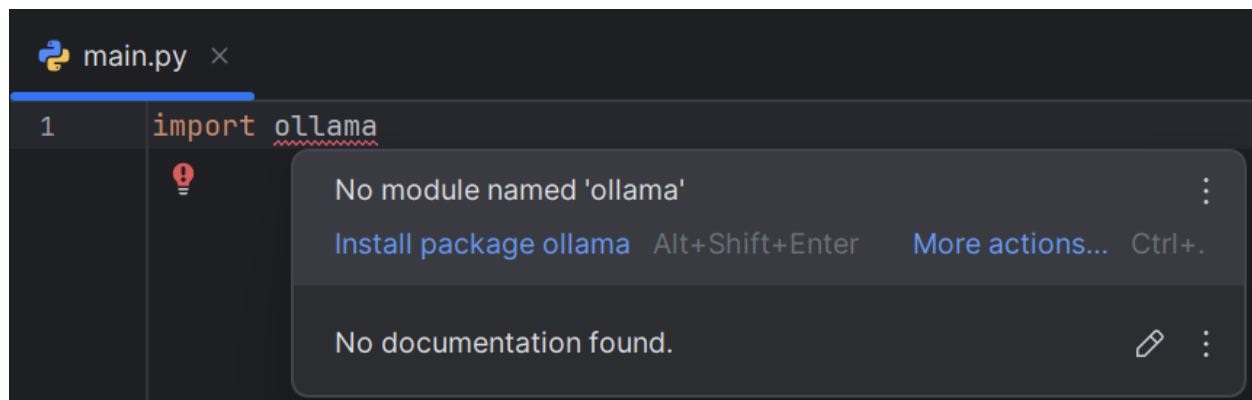
The only Python IDE you need

Built for web, data, and AI/ML professionals.
Supercharged with an AI-enhanced IDE experience.

Download

Free forever, plus one month of Pro included

Using an Integrated Development Environment (IDE) like PyCharm, start a new project with an empty Python script.



To use Ollama in the project, import ollama and click “Install package ollama” which can be seen when hovering over the red error indicator.

```

main.py > ...
1  import ollama
2
3  model = "llama3:8b"
4  system_prompt = "You are a friendly and concise AI assistant. " \
5                  "Answer user questions accurately and keep responses " \
6                  "brief unless the user asks for more detail."
7
8  messages = [{'role': 'system', 'content': system_prompt}] # Conversation history
9

```

With Ollama imported into the project, start by setting up the basic information, including the model's name, system prompt, and the list of messages to track the AI's memory. This list of messages will be sent to the AI each time a new user prompt is provided.

```

11 # Get AI Response
12 response = ollama.chat(model=model, messages=messages)
13 ai_output = response['message']['content'] # Get the plaintext response from the AI
14
15 print("AI Response: " + ai_output)
16

```

The `ollama.chat()` function allows the program to send the conversation to the AI. A response is given as a dictionary where the plaintext response can be found.

```

11 # -- Conversation Loop -- #
12 while True:
13     try:
14         user_prompt = input("Prompt: ")
15         messages.append({'role': 'user', 'content': user_prompt})
16
17         # End Conversation
18         if user_prompt.lower() == "quit":
19             break
20
21         # Get AI Response
22         response = ollama.chat(model=model, messages=messages)
23         ai_output = response['message']['content'] # Get the plaintext response from the AI
24
25         # Update Memory
26         messages.append({'role': 'assistant', 'content': ai_output})
27
28         # Display Response
29         print("AI Response: " + ai_output, end="\n\n")
30     except Exception as e:
31         print("Error", e)
32         break
33

```

Adding the following loop around the chat function allows for back-and-forth communication with the AI.

```
Prompt: Hello! My name is Jonathan
AI Response: Nice to meet you, Jonathan! How can I help or chat with you today?

Prompt: I was hoping you could tell me about Cisco
AI Response: Cisco Systems is a well-known American technology company that specializes in networking hardware, software, and services. They're famous for their routers, switches, and other network equipment. Their products are used by many organizations, including businesses, governments, and educational institutions.

Would you like to know more about specific Cisco products or their role in the industry?
```

Running this program gives you a simple terminal-based chat environment. However, you'll notice that the AI takes a while to respond, but does not indicate that it is working on its response. It is only after it has returned the full response that you know it has generated. This can be fixed by streaming the AI's response as small chunks of text to the terminal as it is being generated.

```
# Get AI Response
print("AI Response: ", end="")

ai_output = ""
response_chunks = ollama.chat(model=model, messages=messages, stream=True)

# Loop through every chunk as it is generated
for chunk in response_chunks:
    text = chunk['message']['content']
    ai_output += text # Save each chunk to a string of the completed AI response
    print(text, flush=True, end="")

# Update Memory
messages.append({'role': 'assistant', 'content': ai_output})

print() # New line
```

Replacing the section of code in the loop to generate the AI response with this new code allows the AI to stream each chunk back and print it immediately to the console. When the stream flag is set to true, the function returns an iterator rather than the final message. This iterator establishes a connection with the model and receives all of the generated chunks of text. In the for loop, these chunks are looped through and added to the console with the print statement. The final message is slowly constructed and added to the memory when it has been completed.

```

server.py > ...
1  from flask import Flask, send_from_directory
2  import os
3
4  app = Flask(__name__)
5
6  # Serve index.html at the root URL
7  @app.route('/')
8  def serve_index():
9      return send_from_directory(os.getcwd(), 'index.html')
10
11 if __name__ == '__main__':
12     # Run the server on all interfaces, port 5000
13     app.run(host='0.0.0.0', port=5000, debug=True)
14

```

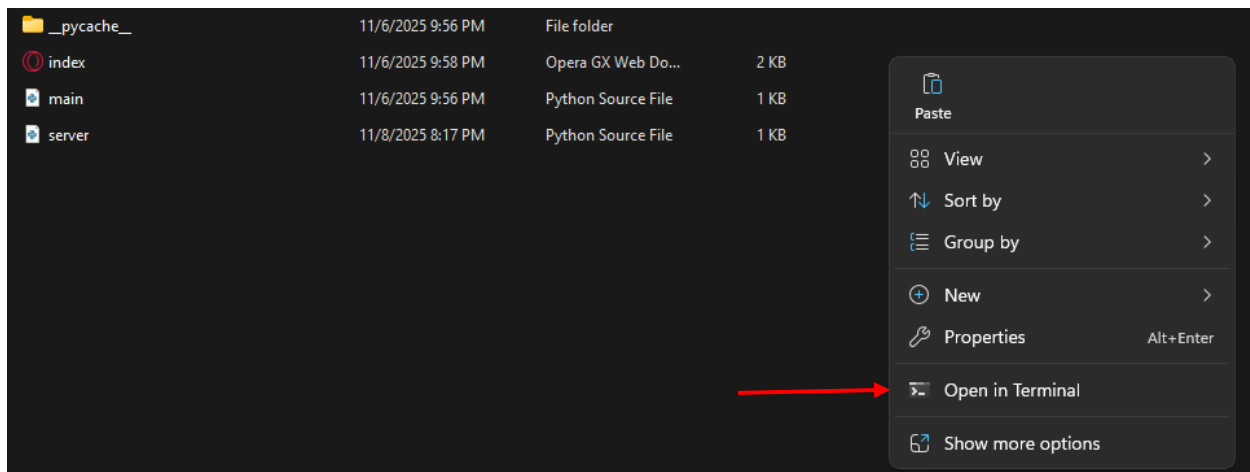
With Flask, you can start up a server using your computer as the host. The IP address 0.0.0.0 allows the server to be accessible by both local and public IP addresses, making the server visible to any device allowed to reach your computer. By default, the website will load the index.html file.

```

index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <body>
4          <h1>AI Webpage</h1>
5          <p>Hello!</p>
6      </body>
7  </html>
8

```

This simple HTML file will load up a page welcoming the user to an empty site. It's not much, but it is a nice placeholder before moving on.



To start up the server, you can press the run button in your IDE, or you can run the script through the command prompt. To do the latter, open the folder containing this project in File Explorer. Here, right-click and select “Open in Terminal.”

```
Microsoft Windows [Version 10.0.26200.7019]
(c) Microsoft Corporation. All rights reserved.

C:\Users\jonat\Documents\AI_Web>python server.py
|
```

In the Command Prompt, type “python server.py.” This will run the script with the server code.



In a web browser, go to your computer's IP address at port 4000. Going to the website, this is what you will see. To implement the Python AI into this webpage, it will require some JavaScript.


```

main.py > ...
1  import ollama
2
3  model = "llama3:8b"
4  system_prompt = (
5      "You are a friendly and concise AI assistant. "
6      "Answer user questions accurately and keep responses "
7      "brief unless the user asks for more detail."
8  )
9
10 messages = [{'role': 'system', 'content': system_prompt}]
11
12 def chat_stream(user_prompt):
13     messages.append({'role': 'user', 'content': user_prompt})
14
15     response_chunks = ollama.chat(model=model, messages=messages, stream=True)
16     ai_output = ""
17
18     for chunk in response_chunks:
19         text = chunk['message']['content']
20         ai_output += text
21         yield text # Stream this back to Flask
22
23     messages.append({'role': 'assistant', 'content': ai_output})
24

```

To make this happen, we will need to update the main.py script that contains the AI functionality. By creating a function that can be called outside of the file, we can talk to the AI from the website. The yield keyword turns the function into a generator function, allowing it to return each chunk generated by the AI over time.

```

server.py > ...
1  from flask import Flask, request, Response, send_from_directory
2  import os
3  from main import chat_stream
4
5  app = Flask(__name__)
6
7  # Serve index.html at the root URL
8  @app.route('/')
9  def index():
10     return send_from_directory(os.getcwd(), 'index.html')
11
12 # SSE endpoint to stream AI responses
13 @app.route('/chat', methods=['POST'])
14 def chat():
15     user_prompt = request.form.get('prompt', '')
16
17     def generate():
18         for chunk in chat_stream(user_prompt):
19             yield chunk
20
21     return Response(generate(), mimetype='text/plain')
22
23 if __name__ == '__main__':
24     # Run the server on all interfaces, port 5000
25     app.run(host='0.0.0.0', port=5000, debug=True)
26

```

Next, in the server.py script, we need to allow the website to find the AI in the project folder, meaning there needs to be a directory associated with the function. With `@app.route`, the chat function can be called and given the parameter for the user prompt.

```

7      <h2>Ask the AI</h2>
8      <form id="chatForm">
9          <input type="text" id="prompt" name="prompt" placeholder="Enter your question..." style="width:300px;">
10         <button type="submit">Send</button>
11     </form>
12
13     <h3>Response:</h3>
14     <pre id="response"></pre>

```

This new HTML in the index.html file creates an input field for the user to submit their prompt, as well as a place for the AI to stream their response back.

```

16 <script>
17   const form = document.getElementById('chatForm');
18   const responseBox = document.getElementById('response');
19
20   form.addEventListener('submit', async (e) => {
21     e.preventDefault();
22     responseBox.textContent = ''; // Clear old response
23     const formData = new FormData(form);
24
25     // Call AI
26     const response = await fetch('/chat', {
27       method: 'POST',
28       body: formData
29     });
30
31     const reader = response.body.getReader();
32     const decoder = new TextDecoder();
33
34     // Stream chunks as they come in
35     while (true) {
36       const { done, value } = await reader.read();
37       if (done) break;
38       responseBox.textContent += decoder.decode(value);
39     }
40   });
41 </script>

```

To deal with functionality on the website, this JavaScript reads the user's prompt and sends it to the directory defined earlier at /chat. From there, the code reads the response as it flows in and adds it to the AI's checkbox. When the next prompt comes through, the AI's previous response is wiped and replaced with the new one. While the old responses do not show up, the AI still has memory saved in the main.py file.

Ask the AI

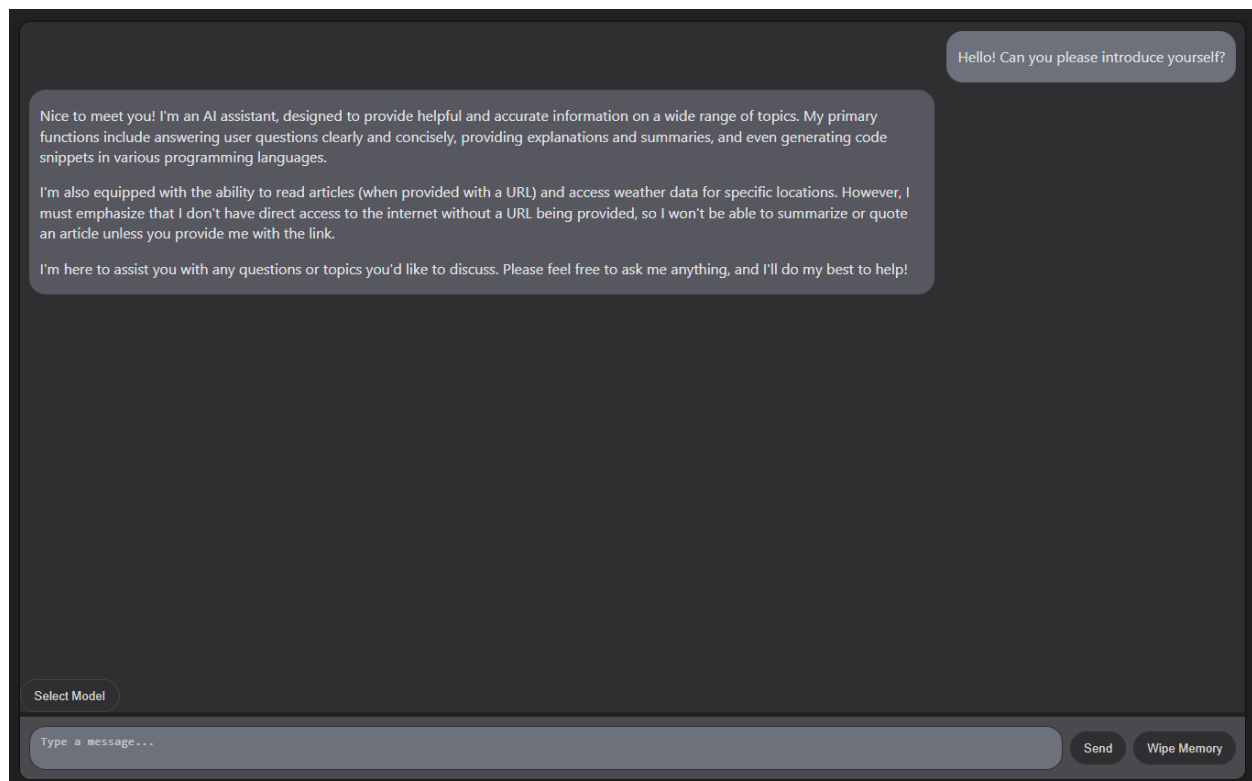
Hello! Please introduce yourself

Send

Response:

Nice to meet you! I'm an AI assistant, designed to provide accurate and helpful information in a quick and easy way. I'm here to assist with any questions or tasks you may have. What's on your mind?

With that, we have a functional AI that can be accessed on any web browser from any device connected to the network. There are, however, a few issues. First, the interface is a little bland. There isn't any CSS to add some style. Second, if two users want to talk to the AI at the same time, they will be sharing memory since all the AI's memory is stored in one chat history list. Furthermore, when the server shuts down, all the memories are lost. Additionally, it would be nice to see old chats in the style of text messages.



With some HTML, CSS, and JavaScript thrown on top, all of those problems can be solved. You can turn this simple script into a fully functional, polished AI chatbot

accessible by a web browser like the one described earlier. However, this one is built from scratch, making it fully customizable and easier to manipulate. Using an SQL server to store the memory for multiple simultaneous users, this functions as a customizable personal AI that can be accessed from anywhere on a network.

Can you help me find the derivative of the following function? $f(x) = \sqrt{x} + 3x^3$

I'd be happy to help!

To find the derivative of $f(x) = \sqrt{x} + 3x^3$, we can use the chain rule and the power rule.

First, let's differentiate the first term, $\sqrt{x}$:

$$\frac{d}{dx}\sqrt{x} = \frac{1}{2}x^{-1/2}$$

Now, let's differentiate the second term, $3x^3$:

$$\frac{d}{dx}(3x^3) = 9x^2$$

So, the derivative of $f(x)$ is:

$$f'(x) = \frac{1}{2}x^{-1/2} + 9x^2$$

With MathJax formatting, the AI can solve math problems and give detailed step-by-step guides so you can solve them yourself.

Can you write me a simple Python calculator? It should be able to add, subtract, multiply, or divide any 2 float values given by the user. Print out the response as a full equation.

Here's a simple Python program that does exactly what you asked for:

```
def calculator():
    num1 = float(input("Enter the first number: "))
    operator = input("Enter the operator (+, -, *, /): ")
    num2 = float(input("Enter the second number: "))

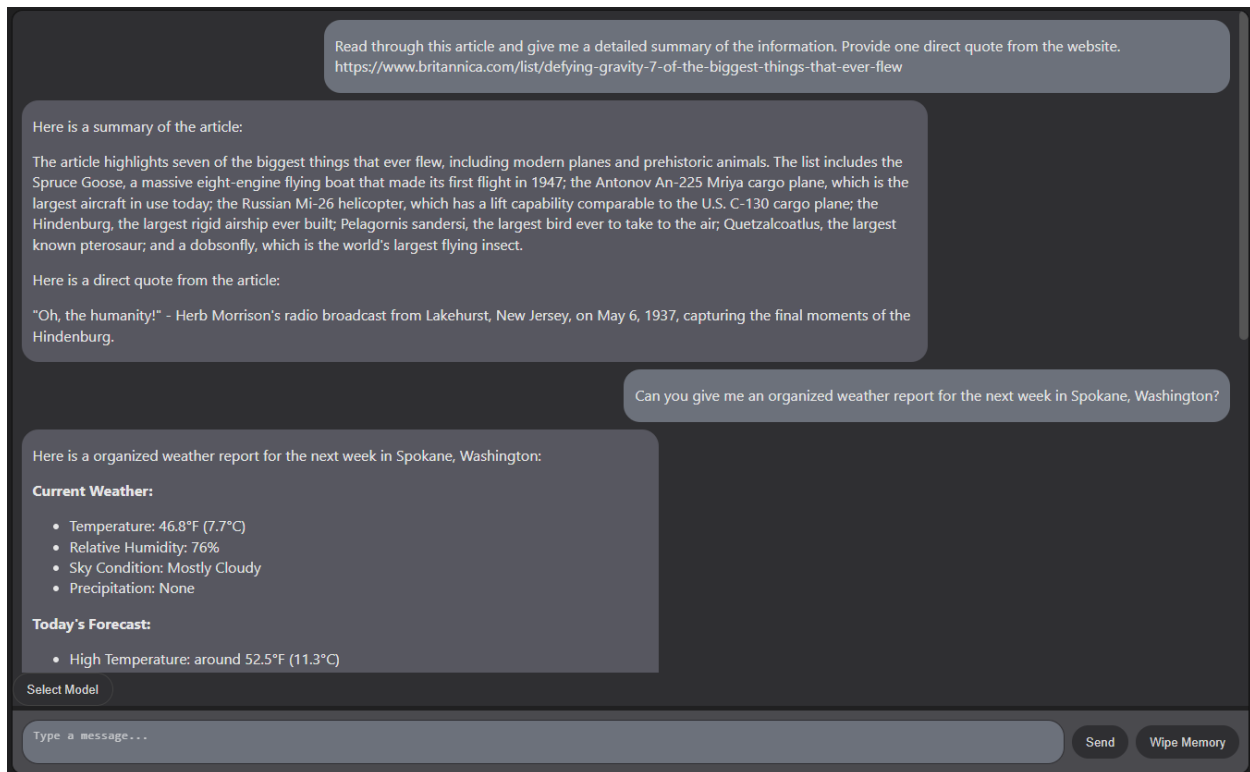
    if operator == "+":
        print(f"{num1} + {num2} = {num1 + num2}")
    elif operator == "-":
        print(f"{num1} - {num2} = {num1 - num2}")
    elif operator == "*":
        print(f"{num1} * {num2} = {num1 * num2}")
    elif operator == "/":
        if num2 != 0:
            print(f"{num1} / {num2} = {num1 / num2}")
        else:
            print("Error! You can't divide by zero.")
    else:
        print("Invalid operator. Please enter +, -, * or /.")

calculator()
```

This program defines a function called calculator that asks the user for two numbers and an operator. It then performs the operation based on the operator chosen and prints out the result in the form of an equation.

Note that I included a check to make sure you're not trying to divide by zero, because that's undefined in mathematics!

Depending on the model, the AI is able to write software using multiple languages, ranging from simple to complex programs.



With access to the Internet, the AI can scrape a variety of webpages and summarize the data, as well as provide accurate and detailed weather reports for nearly any location on Earth.

Problems

When setting up the AI for the web browser with OpenWebUI, I did not have any problems once I learned of the necessary software. Going into this lab, I did not have much experience working with AI. So, I did not know what I needed in order to host an AI model on my computer. After learning about Ollama, Docker, and OpenWebUI, the process went smoothly.

However, when I began developing my own web-based AI through Python and Ollama, I ran into many challenges and bugs.

To have the memory system be unique for each user, I thought of using browser cookies. However, after just a few chats, the AI's memory would quickly use up all the cookies the browser would allow. For a more robust system, I looked into the basics of setting up an SQL server. This proved to be a much better solution.

After that, I started experimenting with having the AI work with the code to make decisions. For example, if the user asked about the weather, the AI should call an API to get weather data about the location they were talking about, then summarize the data accordingly. However, to do so, I needed a separate AI model dedicated to making these decisions and writing responses in the format of a JSON

file. This secondary analysis AI worked well initially. When I asked about the weather, the AI would see this and write a JSON file that the code could read. However, after experimenting with longer conversations, I noticed that the analysis AI began ignoring its instructions to answer only in a JSON format, and it began to respond as though the user could see its responses. Essentially, it wanted to join in the conversation. In reality, the AI cannot be seen by the user, and the system prompt makes this clear. It is only there to analyze the user's prompt and decide if an action needs to take place within the code. Ollama has a built-in feature called "tools" which would solve my problem, but I could not find a model of a reasonable size that supported this feature. So, I decided to rely on keyword-based function routing, where the program searches for keywords to determine a proper response. This was not perfect, so I used the AI as a backup. Instead of trying to determine what function to call, the AI is told to summarize the intent of the user, making sure to include important details and keywords in every response. Then, this intent summary, along with the original prompt, is run through keyword analysis. To ensure that the intent summary AI does not forget its job, I prefaced each user prompt run by this AI with the request to summarize the intent of the following prompt with respect to the system prompt. Overall, this proved to be a much more reliable method, allowing me to expand on the number of functions that could be called, such as web scraping.

Conclusion

Ultimately, locally hosting AI is an easy process. With modern software, it requires just a few clicks to get an AI up and running on any computer. Then, making this AI accessible to anyone in the network through a web browser has been made easy with OpenWebUI. While running powerful models requires powerful hardware, the control and security that local AI provides is remarkable. With private data staying private, speeds beyond what remote AI could provide, no per-usages charges, and complete customization with a little bit of code, local AI hosting is a suitable solution for anyone requiring the processing power and intelligence of modern AI.